

Claude Skills desde cero

Crea tu primer skill personalizado en 30 min

Una guía práctica de Decodifica · decodifica.net

Por Jordi · [@decodificaia](https://twitter.com/decodificaia)

Indice

#	Seccion	Subtemas
1	Qué es un Claude Skill (y qué no es)	4
2	Cuándo Sí vale la pena crear un skill	4
3	Estructura de un skill	5
4	Ejemplo completo: analizador de emails	4
5	Errores típicos al crear skills	4
6	Workflow recomendado: crear tu primer skill	5
7	Recursos para profundizar	5

Este PDF es el segundo lead magnet de Decodifica. Te enseño a crear Claude Skills desde cero: qué son, cuándo vale la pena, cómo se estructuran, y un ejemplo real que uso a diario. Todo basado en mi experiencia creando 8 skills en los últimos meses.

1. Qué es un Claude Skill (y qué no es)

Un skill NO es un plugin, una app, ni un agente. Es un paquete de instrucciones + scripts que Claude carga automáticamente cuando detecta que el tema es relevante. Piensa en "knowledge packs" especializados que extienden lo que Claude sabe hacer sin tener que entrenar un modelo nuevo.

Diferencia con un prompt largo

Un prompt largo lo pegas en cada conversación y Claude lo lee una vez. Un skill es un archivo .md (o carpeta con varios) que Claude descubre y aplica automáticamente. Tú no le dices "usa este skill", él decide cuándo basándose en la descripción del skill y el contexto de la conversación.

Diferencia con un agente o GPT custom

Los GPT custom de OpenAI y los agentes son más "pesados" - tienen su propia UI, sus propios límites, su propia memoria. Un skill es ligero: un archivo markdown + opcionalmente código. Lo guardas en una carpeta, lo subes a Claude, y funciona. Sin backend, sin deploy, sin servidor.

Lo que un skill SÍ hace bien

Estandarizar un workflow repetitivo (ej. "cada vez que el usuario pida X, haz Y siguiendo estos pasos"). Añadir conocimiento específico (ej. "el tono de mi marca", "el schema de mi DB"). Encadenar tools existentes en una secuencia concreta. Cargar contexto pesado solo cuando es relevante (ahorra tokens en conversaciones que no lo necesitan).

Lo que un skill NO hace bien

No ejecuta código de producción - el código del skill corre en local con MCP servers o similar, no en el sandbox de Claude. No reemplaza un agente autónomo - un skill es pasivo, espera a que el tema aparezca. No es un sistema RAG completo - un skill inyecta contexto en el system prompt, no hace búsqueda semántica sobre tus documentos.

2. Cuándo SÍ vale la pena crear un skill

He creado 8 skills en los últimos 3 meses. 5 de ellos los uso CADA DÍA. Los otros 3 fueron pérdida de tiempo. La diferencia: los útiles resuelven un problema recurrente, los inútiles eran para algo que solo hice una vez.

Señales de que vale la pena

1) Haces el mismo tipo de tarea más de 3 veces por semana. 2) Cada vez que lo haces, pegas el mismo bloque de instrucciones. 3) La calidad varía mucho según si te acuerdas o no de incluir X detalle. 4) Otras personas del equipo también podrían usarlo.

Señales de que NO vale la pena

1) Es algo que haces una vez al trimestre. 2) Las instrucciones cambian cada vez. 3) Es tan complejo que el skill acabaría siendo un manual. 4) Solo lo usarías tú y no quieres mantenerlo.

Regla del "1 minuto de setup, 5 minutos ahorrados por uso"

Si tardas más de 1 hora en crear el skill (incluyendo tests) y no lo vas a usar 60 veces, no compensa. Los skills buenos cuestan 30-45 min de creación y se usan cientos de veces.

Mi caso real: el skill "analizador de emails"

Tardé 40 min en crearlo. Lo uso 5-8 veces al día (cada email que recibo, lo paso por Claude con el skill activo). ROI brutal. Sin el skill, tenía que pegar el mismo prompt de 200 palabras cada vez y rezar para no olvidar el "no inventes" o el "tono ejecutivo".

3. Estructura de un skill

Un skill mínimo es UN archivo markdown con frontmatter YAML. Un skill complejo es una carpeta con SKILL.md + scripts opcionales + referencias opcionales. Empieza siempre con el mínimo y añade complejidad solo cuando la necesites.

SKILL.md mínimo (10 líneas)

Un skill con la estructura mínima tiene: nombre en el frontmatter (campo ``name``), descripción de qué hace y cuándo se activa (campo ``description``), y el cuerpo del skill con las instrucciones que Claude va a leer. Eso es todo.

Frontmatter YAML obligatorio

Los dos campos clave son ``name`` y ``description``. El name es el identificador interno. La description es CRÍTICA: Claude decide si carga el skill basándose en ella. Una description vaga = skill inútil. Una description específica = skill activado cuando toca.

Ejemplo real de frontmatter

```
--- name: email-analyzer description: Analiza emails entrantes y extrae tareas, fechas límite, personas mencionadas y nivel de urgencia. Úsalo cuando el usuario pegue un email y pida extraer info, resumir, o convertirlo en tareas para un sistema externo. ---
```

Cuerpo del skill: instrucciones

Después del frontmatter viene markdown normal. Escribe como si le explicarás a un colega nuevo qué tiene que hacer con esta información. Sé específico, da ejemplos, pon reglas claras ("no inventes", "si no estás seguro, pregunta").

Cuándo añadir scripts (vía MCP)

Si el skill necesita ejecutar código (leer archivos, llamar APIs, hacer queries), lo haces vía un MCP server. Pero empieza SIN scripts. El 80% de los skills útiles son solo markdown. Los scripts añaden complejidad de deploy y mantenimiento.

4. Ejemplo completo: analizador de emails

Este es un skill que yo uso a diario. Cópialo, adapta los campos, úsalo. Es el ejemplo más realista que puedo darte porque lo tengo activo y funcionando.

SKILL.md completo

```
---
name: email-analyzer
description: Analiza emails entrantes. Extrae: remitente, asunto, fecha, tareas implícitas, fechas límite, personas mencionadas, nivel de urgencia (1-5), y si requiere respuesta. Devuelve JSON estructurado. Úsalo cuando el usuario pegue texto de un email y pida extraer info, tareas, o estructurar para un sistema (Notion, Trello, calendar).
---
```

Email Analyzer

Cuando el usuario pegue un email, devuélvelo en este formato JSON:

```
{
  "remitente": "string",
  "asunto": "string",
  "fecha_recibido": "ISO 8601",
  "personas_mencionadas": ["string"],
  "tareas": [
    {
      "descripcion": "string",
      "responsable": "string",
      "fecha_limite": "ISO 8601 o null",
      "prioridad": "alta | media | baja"
    }
  ],
  "nivel_urgencia": 1-5,
  "requiere_respuesta": true | false,
  "razon_urgencia": "string"
}
```

Reglas:

- Si no encuentras un campo, ponlo como null. NO inventes.
- Las tareas implícitas cuentan: "podrías echarle un vistazo al X" es una tarea aunque no esté escrita como imperativo.
- Las fechas en español ("para el viernes", "antes del 15") interprétalas a ISO 8601 ("YYYY-MM-DD"). Si no puedes deducir la fecha, null.
- El nivel de urgencia 5 = bloqueante hoy, 1 = informativo para cuando sea.
- "requiere_respuesta" = true si el email te pide algo, te hace una pregunta, o te asigna algo.

Cómo lo activo

Claude carga el skill automáticamente cuando detecta un email en la conversación Y la description del skill lo menciona. La description dice "cuando el usuario pegue texto de un email" - eso es la señal. Yo

solo pego el email y digo "extrae las tareas". Claude sabe qué hacer.

Por qué JSON estructurado

El JSON permite que después pase el output a un sistema (Notion API, Trello, calendar) sin tener que parsear texto libre. Es la diferencia entre "tenemos que revisar el contrato para el viernes" y {descripcion: "Revisar contrato", fecha_limite: "2026-06-13", prioridad: "alta"}.

Adaptar a tu workflow

Cambia el JSON al schema de tu sistema. Si usas Trello, los campos son listas (list_id, due_date, labels). Si es Notion, son properties con tipos. El skill es la lógica de extracción, el schema es tu decisión.

5. Errores típicos al crear skills

8 skills creadas, 5 útiles, 3 fracasos. Estos son los 3 errores que me costaron tiempo:

Description vaga = skill ignorado

Mi primer error: escribir description tipo "skill para ayudar con emails". Claude nunca sabía si activarlo. Lo cambié a "Analiza emails entrantes y extrae tareas. Úsalo cuando el usuario pegue un email y pida extraer info" y empezó a activarse siempre. LA DESCRIPTION ES LA INTERFAZ CON CLAUDE.

Instrucciones vagas = skill inestable

"Resume este email" produce resúmenes inconsistentes. "Extrae remitente, fecha, tareas, urgencia en formato JSON con estas reglas" produce siempre lo mismo. Cuanto más específico el prompt, más estable el output.

Skill demasiado ambicioso

Mi tercer error: un skill que hacía de todo (resumir emails, analizarlos, generar respuestas, programar follow-ups). Pesaba 800 líneas, Claude se perdía, el resultado era mediocre. Lo partí en 3 skills: analizar, responder, programar. Cada uno de 150 líneas, mucho más efectivo.

No probar con casos reales

Otro error: diseñar el skill en abstracto, escribirlo "perfecto", y luego probarlo con un email real y descubrir que faltaba el caso de "el email es un reply con quote del email anterior". SIEMPRE probar el skill con 5-10 casos reales antes de declararlo listo.

6. Workflow recomendado: crear tu primer skill

Si nunca has creado un skill, este es el orden de pasos que te funciona. Es el mismo que usé yo para los 5 que mantengo activos.

Paso 1 - Identifica el patrón

Durante 1 semana, anota cada vez que pegas el mismo bloque de instrucciones en un prompt. Si lo haces 3+ veces en 7 días, tienes candidato a skill.

Paso 2 - Escribe el SKILL.md mínimo

Solo frontmatter (name + description) + cuerpo con las instrucciones. 30-45 min máximo. NO añadas scripts, NO añadas referencias, NO intentes hacerlo perfecto a la primera.

Paso 3 - Pruébalo con 5 casos reales

Coge 5 ejemplos del mundo real (no inventados) y prueba el skill. Anota dónde falla. Itera la description y las instrucciones hasta que 4/5 casos funcionen bien.

Paso 4 - Mide el ROI

Si después de 2 semanas no lo has usado 10+ veces, bórralo. Si lo usas a diario, considera añadir un script o referencia para hacerlo más potente.

Paso 5 - Documenta y comparte

Una vez estable, documéntalo en un README con: qué hace, cuándo usarlo, ejemplo de output. Si lo usas con el equipo, compártelo. Si lo usas solo, con el README en tu repo de skills basta.

7. Recursos para profundizar

Claude Skills es una funcionalidad nueva y la documentación oficial está en construcción. Estos son los recursos que me han servido:

Docs oficiales de Claude Skills

<https://docs.claude.com/en/docs/agents-and-tools/agent-skills/overview> - actualizado por Anthropic. Incluye el spec de SKILL.md, ejemplos canónicos, y best practices.

Repo de skills de la comunidad

github.com/anthropics/skills - skills open source oficiales y de la comunidad. Buen lugar para inspirarte y forkear.

Mi blog post completo

<https://decodifica.net/blog/claude-skills-desde-cero/> - versión extendida de este PDF, con más ejemplos, pitfalls, y el código del MCP server que uso para mis skills con scripts.

Cursos y workshops

Anthropic ha empezado a hacer workshops sobre Skills. Búscalos en su canal de YouTube oficial. Duran 1-2 horas, muy recomendables para ir más allá del SKILL.md básico.

Comunidad Discord

Hay un Discord no oficial de power users de Claude donde comparten skills y patrones. Búscalo en el subreddit [r/ClaudeAI](https://www.reddit.com/r/ClaudeAI).

Gracias por leer

¿TE HA GUSTADO ESTE PDF?

Si quieres recibir uno nuevo cada sábado, suscríbete a la newsletter:
<https://buttondown.com/decodifica>

Para más tutoriales sobre Claude, IA y productividad:
<https://decodifica.net/blog>

Y si creas tu primer skill, mandame el repo y lo pruebo.